Manual Técnico

Proyecto2

José Fernando Calderón Castillo 202330678

Introducción

Uno de los clientes más importantes de Code & Bugs es un gran fan de la franquicia de juegos Plantas vs. Zombis, sin embargo, los zombis siempre fueron sus personajes preferidos, por ello ha solicitado a los mejores programadores de Code & Bugs, construir un remake del juego llamado Zombis vs. Plantas, en el que él pueda estrategizar la partida desde el lado de los zombis y vencer a las plantas para lograr conseguir el cerebro protegido por la computadora.

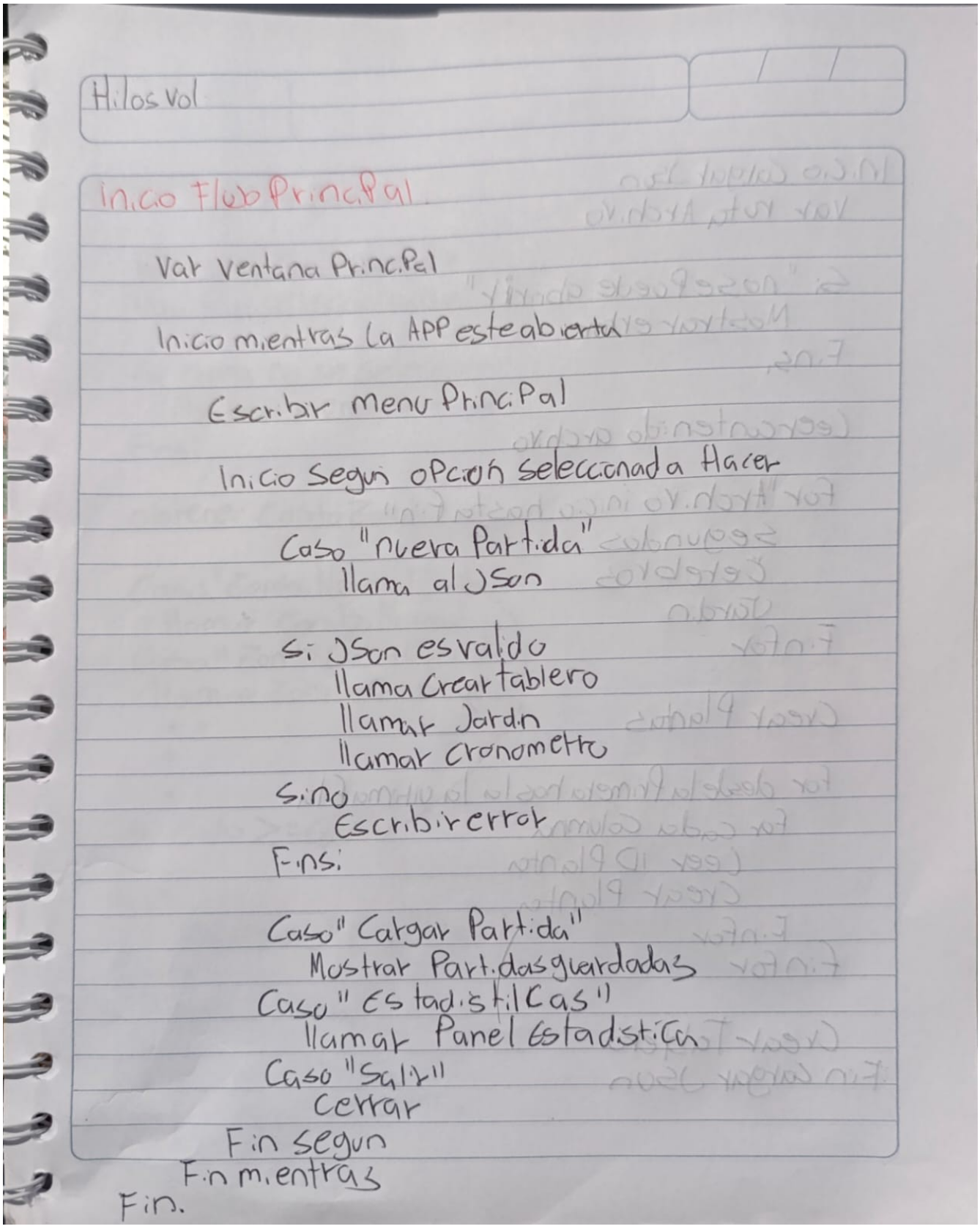
Tecnologías utilizadas

En este proyecto se utilizo la versión de Java JDK 21 la cual dejo el link de descarga: https://www.oracle.com/java/technologies/javase/jdk21-archive-downloads.html El IDE utilizado fue Neatbeans 25 el cual dejo el link de descarga aquí: https://netbeans-apache-org.translate.goog/front/main/download/nb25/? _x_tr_sl=en&_x_tr_tl=es&_x_tr_hl=es&_x_tr_pto=tc

El sistema de construcción fue Maven

Algoritmos principales / diseño del programa de la practica 1

Pseudo código del menú:



```
Hilos Vol

Inicio FlebPrincipal

    Var ventana PrincPal

    Inicio mientras La APP este abierta

        Escribir menu Principal

        Inicio Segun opción seleccionada Hacer

            Caso "nueva Partida"
                llama al JSon

            si JSon es valido
                llama Crear tablero
                llamar Jardin
                llamar cronometro
            sino
                Escribir error
            Fin si

            Caso" Cargar Partida"
                Mostrar Partidas guardadas
            Caso " Estadistilcas"
                llamar Panel Estadistica
            Caso "Salir"
                cerrar
        Fin segun
    Fin mientras
Fin.
```

```
Inicio Catgar Json
  Var ruta Archivo

  Si "noSePuede abrirlt"
     Mostrar error
  Fins,

  Leer contenido archivo

  For "Archivo inicio hasta Fin"
      Segundos
      Cerebros
      Jardin
  Fin for

  Crear Plantas

  For desde la Primera hasta la la ultima fila
     For cada Columna
        Leer ID Planta
        Creat Planta
     Fin for
  fin for

  Creat Tablero
Fin Catgar Json
```

Inicio IntroCatZombi

Var ZombiSeleccionado
Var FilaseleccionadaS.

Si Zombi no se selecciona.
  Mostrar mensaje
Finsi

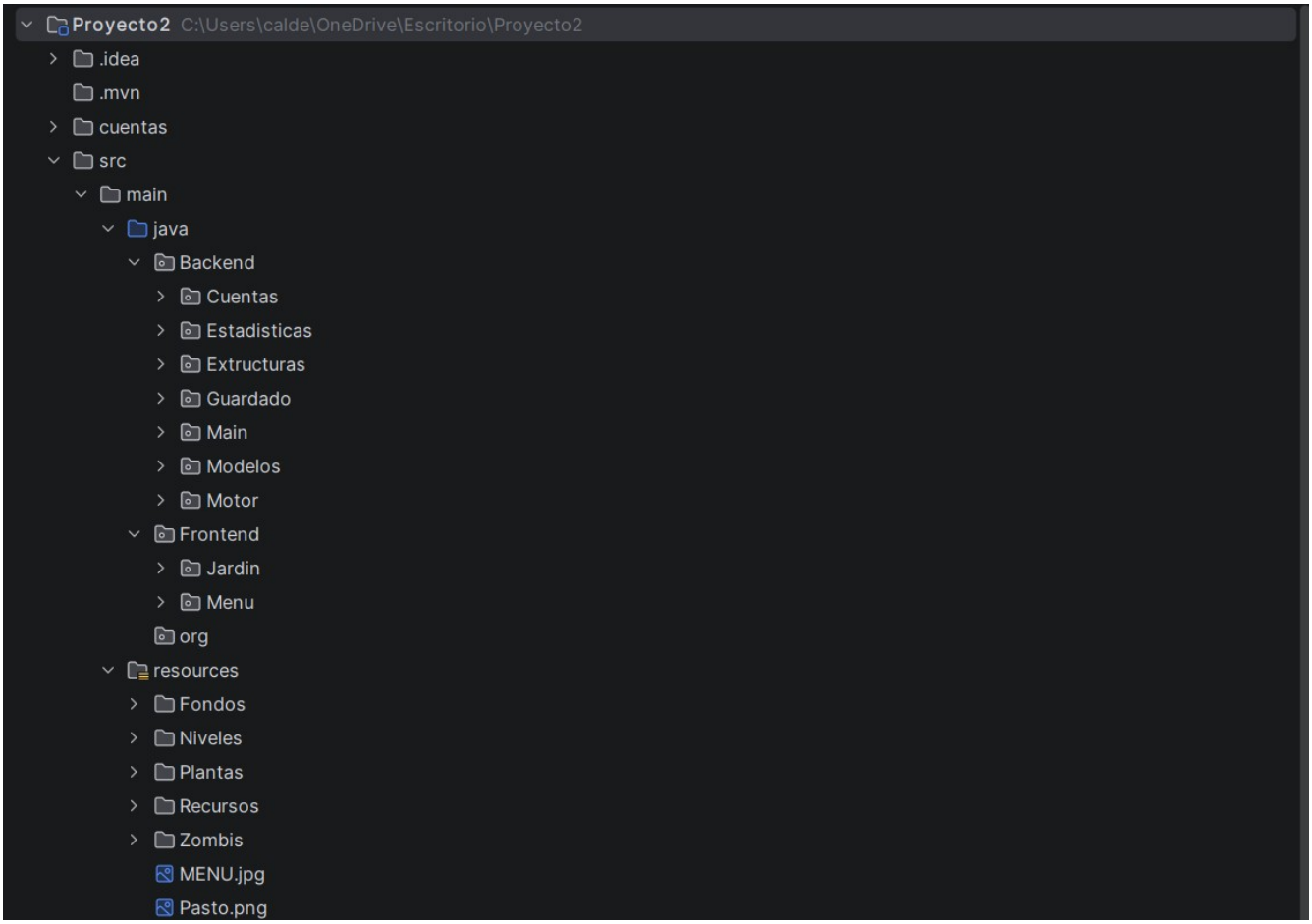
obtener costoZombi.

Caso "ZombiNormal"
  llamar Zomb.Normal
Caso "ZombiCaraCono"
  llamar ZombiCaraCono
  .
  .
  .
Si costo >= Costo

Sino
  Mostrar "Cerebros insuficientes"
Finsi
Fin.

Datos técnicos de la practica:

Estructura del proyecto:



Debido a que esta practica esta enfocada a la practica de Programación Orientada a Objetos, Swing y Archivos binarios dividimos todas las clases por carpetas, se clasificaron por tener características similares o por que heredan de si mismas.



Por Swing dividimos todo por Backend y Frontend, Backend es toda la lógica detrás del juego, mientras que el Fronted es lo que miramos



El Bakend Tiene carpetas las cuales sirven para que toda la lógica del juego siga



La carpeta Cuentas es la que esta enfocada en guardar los datos de las partidas, leer los archivos Json que es de donde viene la estructura del Jardín.



Estadísticas esta encargado de gestionar las estadísticas de manera correcta



Estructuras es muy importante por que esta es la encargada de almacenar a listas Dinámicas, que es un método auxiliar para la creacion de matrices con valor variable para no usar Arraylist



La carpeta Guardado es donde se almacenan las clases encargadas en guardar y cargar nuestro progreso



Main es la clase mas importante ya que usamos esta para poder ejecutar nuestro programa



Modelos es una de las mas importantes ya que aquí se encuentran todas las entidades, o criaturas.



Tenemos a los guisantes como un Entidad devido a que estos tienen atributos



La carpeta plantas es la encargada de almacenar la información de todas las plantas



La carpeta de Zombis es la que almacena a cada uno de los zombis



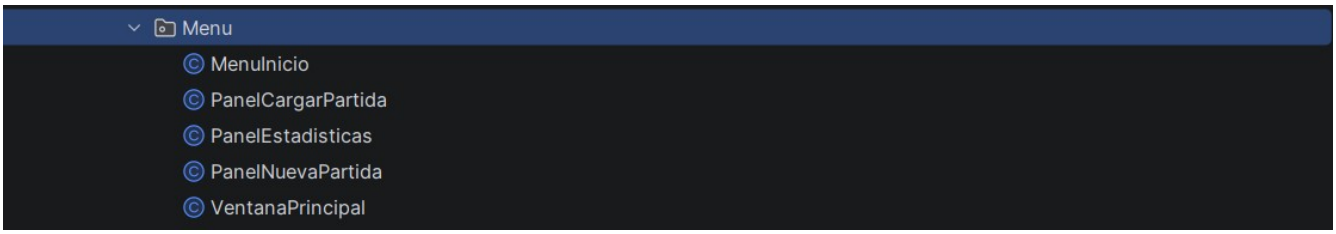
El motor es la segunda clase masa importante ya que aquí se maneja toda la lógica detrás del juego



El Fronted es donde se almacena todo lo visual por lo que se le dividirá en dos escenarios, el primero es en la partida que es donde pasara el 90 % y menú es donde nosotros tomaremos las decisiones como crear una nueva partida o de cargar una.



Aquí en Jardín es donde nosotros almacenamos las clases que vamos a usar para la visualización del Jardín



Menú es donde se almacenan los menús



Y recursos que estas nos servirán para almacenar las imágenes y los archivos externos

Explicación del código

```java
package Backend.Main;
// importo la carpeta fronted ya que ahi se encuentra el menu
import Frontend.Menu.VentanaPrincipal;

public class Main {  Radhdeat
    public static void main(String[] args) {  Radhdeat

        // LLamamos al menu
        VentanaPrincipal ventana = new VentanaPrincipal();
        // para ser mas especificos al metodo para hacer visible el menu
        ventana.setVisible(true);
      }
}
```

Para el main utilizamos la estructura del POO es decir que solo lo utilizamos para llamar otra clase la cual esta encargada para lógica del juego

```java
// Indicamos que la clase principal es un JFrame
public class VentanaPrincipal extends JFrame {  17 usages  Radhdeat

    // Indicamos la cuenta que estara activa
    private String cuentaActual;  2 usages

    // Clase para la parte visual del menu
    public VentanaPrincipal() {  1 usage  Radhdeat
        // Se le asigna un titulo a la pestaña de java
        setTitle("Zombies vs Plantas");
        // se le asigna un tamaño
        setSize( width: 1000,  height: 600);
        // sirve para cuando presionemos x se cierre el juego
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        // es para poder centrar la pantalla
        setLocationRelativeTo(null);
        // Para que el usuario no pueda agrandar la pantalla
        setResizable(false);

        // Se llama al menu
        MenuInicio panelMenu = new MenuInicio( ventana: this);
        cambiarPantalla(panelMenu);
    }
```

La ventana principal es la que usamos para la creación de la interfaz gráfica de una aplicación de Java, es donde asignamos los atributos de dicha ventana, como el titulo el tamaño, pero ojo usamos un codigo setDefaultCloseOperation(Jframe.EXIT_ON_CLOSE) que sirve para que cuando se presiones x se cierre la app.

```java
    // Método para reemplazar el panel visible en la ventana
    public void cambiarPantalla(JPanel nuevoPanel) {  13 usages  Radhdeat
        // Indica cual sera el panel activo
        setContentPane(nuevoPanel);
        // Indica que tiene que revisar la distribucion de los elementos
        revalidate();
        // volvemos a imprimir la interfaz grafica
        repaint();
    }

    // Getters y Setters para controlar el usuario activo en toda la sesión
    public void setCuentaActual(String cuentaActual) { this.cuentaActual = cuentaActual; }

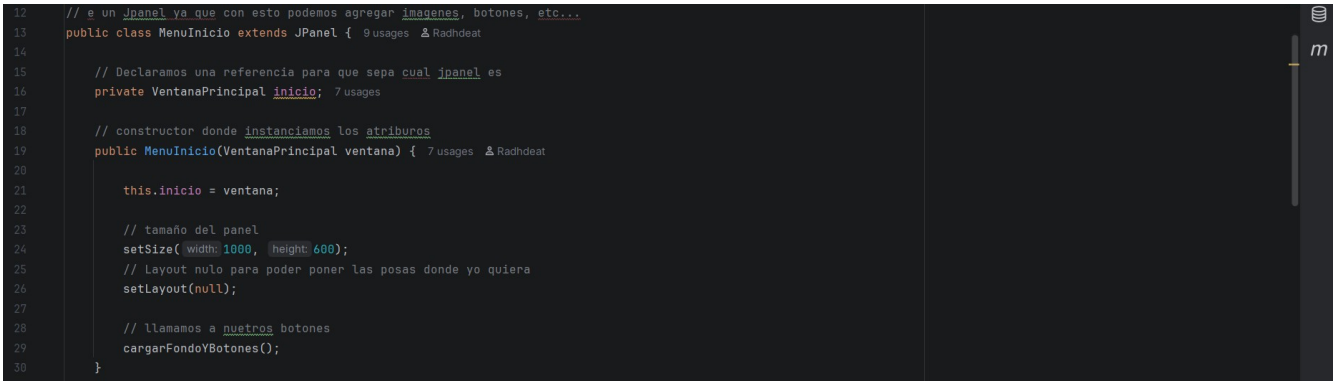
    public String getCuentaActual() { return cuentaActual; }
}
```

Método para que se pueda cambiar la pantalla donde se remplaza el panel actual.

```java
package Frontend.Menu;

// importamos la extension de Swing
import javax.swing.*;
// importamos cosas como el cursor o imagnes
import java.awt.*;
// importamos la calse que detecta acciones como un click
import java.awt.event.ActionEvent;
// importamos la clase que se encarga de ejecutar la accion de un botton
import java.awt.event.ActionListener;
```

Aquí importamos cosas importantes para trabajar Swing y para poder usar click es decir darle acción al cursor



```java
// e un jpanel ya que con esto podemos agregar imagenes, botones, etc...
public class MenuInicio extends JPanel {  9 usages  Radhdeat

    // Declaramos una referencia para que sepa cual jpanel es
    private VentanaPrincipal inicio;  7 usages

    // constructor donde instanciamos los atributos
    public MenuInicio(VentanaPrincipal ventana) {  7 usages  Radhdeat

        this.inicio = ventana;

        // tamaño del panel
        setSize( width: 1000,  height: 600);
        // Layout nulo para poder poner las posas donde yo quiera
        setLayout(null);

        // llamamos a nuetros botones
        cargarFondoYBotones();
    }
```

Usamos un constructor para instanciar los atributos del panel donde declaramos el tamaño y declaramos el Layout nulo para poner las cosas en donde yo quiera atravez de codigo



```java
    // metodo enfocado en poner botnes y fondo
    private void cargarFondoYBotones() {  1 usage  Radhdeat

        // la diagonal en el nombre indica que hara la busqueda desde la raiz de recursos
        java.net.URL ruta = getClass().getResource( name: "/MENU.jpg");

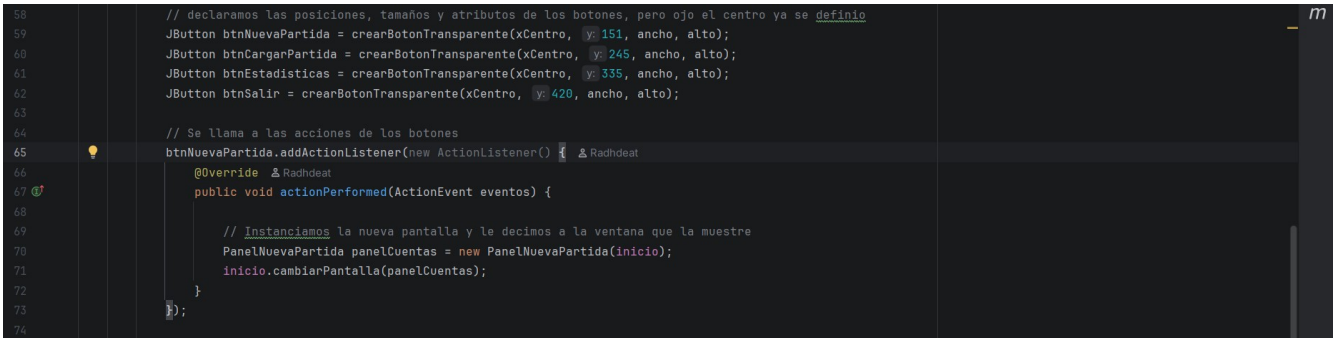
        // si la ruta si existe se genera la imagen
        if (ruta != null) {

            // Trabajamos la imagenes como icocnos y esta busca la imagen desde la ruta
            ImageIcon iconoOriginal = new ImageIcon(ruta);

            /*
            / recibe la imagne y le asigna un tamaño
            / Image.SCALE_SMOOTH solicita un escalado suave para intentar conservar una apariencia visual de mejor calidad.
            */

            Image imgEscalada = iconoOriginal.getImage().getScaledInstance( width: 1000,  height: 600, Image.SCALE_SMOOTH);
            // usamos un jlabel para poder mostrar la imagen
            JLabel fondo = new JLabel(new ImageIcon(imgEscalada));
            // se le asigna una posicion
            fondo.setBounds( x: 0,  y: 0,  width: 1000,  height: 600);
```

Usamos algo importante lo cual es importar una imagen para el fondo, esto lo hacemos atravez de un java.net.URL ruta el cual accede a la carpeta recursos que esta en la raiz



```java
            // declaramos las posiciones, tamaños y atributos de los botones, pero ojo el centro ya se definio
            JButton btnNuevaPartida = crearBotonTransparente(xCentro,  y: 151, ancho, alto);
            JButton btnCargarPartida = crearBotonTransparente(xCentro,  y: 245, ancho, alto);
            JButton btnEstadisticas = crearBotonTransparente(xCentro,  y: 335, ancho, alto);
            JButton btnSalir = crearBotonTransparente(xCentro,  y: 420, ancho, alto);

            // Se llama a las acciones de los botones
            btnNuevaPartida.addActionListener(new ActionListener() {  Radhdeat
                @Override  Radhdeat
                public void actionPerformed(ActionEvent eventos) {

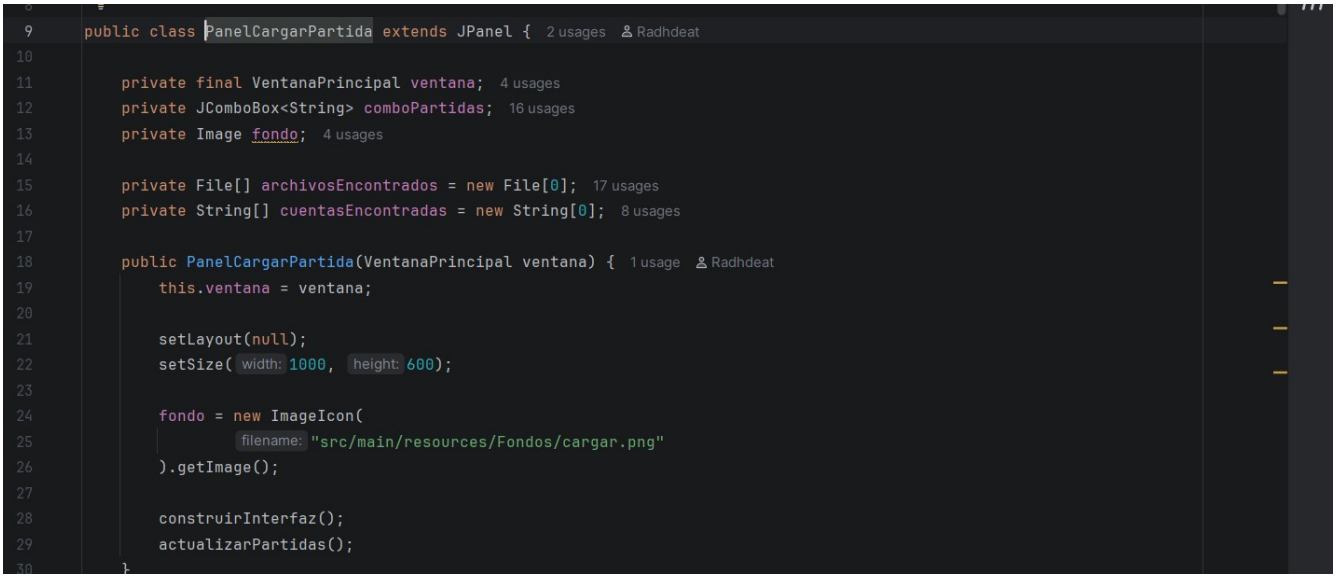
                    // Instanciamos la nueva pantalla y le decimos a la ventana que la muestre
                    PanelNuevaPartida panelCuentas = new PanelNuevaPartida(inicio);
                    inicio.cambiarPantalla(panelCuentas);
                }
            });
```

Cremos a los botones y creamos una clase que ejecuta las acciones de los botones



```java
    private JButton crearBotonTransparente(int x, int y, int ancho, int alto) {  4 usages  Radhdeat
        // Ponesmos los atributos de cada batton
        // creamos un boton sin texto
        JButton btn = new JButton();
        // dejamos las cordenadas como parametros
        btn.setBounds(x, y, ancho, alto);
        // se imprime sin un fondo
        btn.setOpaque(false);
        // se evita que cree un area así no lo pone con un color celeste
        btn.setContentAreaFilled(false);
        // quitamos el borde
        btn.setBorderPainted(false);
        // evita que se imprima el indicador
        btn.setFocusPainted(false);
        // Cambiamos el cursor
        btn.setCursor(new Cursor(Cursor.HAND_CURSOR));
        return btn;
    }
}
```

Clase importante ya que usamos una imagen de fondo pero esta ya tiene los botones sobrepuestos, entonces nosotros creamos botones con atributos que los hacen transparentes así funcionan y se ven bien



```java
public class PanelCargarPartida extends JPanel {  2 usages  Radhdeat

    private final VentanaPrincipal ventana;  4 usages
    private JComboBox<String> comboPartidas;  16 usages
    private Image fondo;  4 usages

    private File[] archivosEncontrados = new File[0];  17 usages
    private String[] cuentasEncontradas = new String[0];  8 usages

    public PanelCargarPartida(VentanaPrincipal ventana) {  1 usage  Radhdeat
        this.ventana = ventana;

        setLayout(null);
        setSize( width: 1000,  height: 600);

        fondo = new ImageIcon(
                filename: "src/main/resources/Fondos/cargar.png"
        ).getImage();

        construirInterfaz();
        actualizarPartidas();
    }
```

Clase encargada de la parte grafica para poder cargar una partida existente, tambien utilizamos fondos con los botones visibles para ya solo usar botones transparentes

```java
private void cargarSeleccionada() {  1 usage  Radhdeat *

    int indice = comboPartidas.getSelectedIndex();

    if (!comboPartidas.isEnabled()
            || indice < 0
            || indice >= archivosEncontrados.length) {

        JOptionPane.showMessageDialog(
                parentComponent: this,
                message: "No hay ninguna partida válida para cargar."
        );
        return;
    }

    File archivo = archivosEncontrados[indice];
    String cuenta = cuentasEncontradas[indice];

    if (!archivo.exists() || !archivo.isFile()) {

        JOptionPane.showMessageDialog(
                parentComponent: this,
                message: "El archivo seleccionado ya no existe."
        );

        actualizarPartidas();
        return;
    }
```

Utilizamos un metodo para la poder cargar partidas seleccionadas pero esta utiliza rutas reales por lo cual vincula y reconstruye las partidas previas

```java

public class PanelEstadisticas extends JPanel {  2 usages  Radhdeat
    private final VentanaPrincipal ventana;  4 usages
    private final ManejadorEstadisticas manejador = new ManejadorEstadisticas();  4 usages
    private final Image imagenFondo;  4 usages
    private JLabel lblPartidasJugadas, lblPartidasGanadas, lblPartidasPerdidas, lblPlantasDestruidas;  3 usages
    private JLabel lblZombisInvocados, lblCerebrosGastados, lblMejorTiempo, lblPodadoras;  3 usages
    private final JLabel[] lblTopRapidos = new JLabel[5];  2 usages
    private final JLabel[] lblTopDestructores = new JLabel[5];  2 usages
    private final JLabel[] lblTopExitosos = new JLabel[5];  2 usages

    public PanelEstadisticas(VentanaPrincipal ventana) {  1 usage  Radhdeat
        this.ventana = ventana;
        setLayout(null); setSize( width: 1000,  height: 600);
        imagenFondo = new ImageIcon( filename: "src/main/resources/Fondos/estadisticas.png").getImage();
        construirInterfaz();
        cargarDatosReales();
    }
```

Para las estadísticas también utilizamos un fondo con los espacios donde se colocara la información, por lo que hacemos los cuadros de textos invisibles pero el texto si es visible

```java
private void construirInterfaz() {  1 usage  Radhdeat
    lblPartidasJugadas = crearLabelDato( x: 340, y: 150, ancho: 175, alto: 28);
    lblPartidasGanadas = crearLabelDato( x: 340, y: 185, ancho: 175, alto: 28);
    lblPartidasPerdidas = crearLabelDato( x: 340, y: 220, ancho: 175, alto: 28);
    lblPlantasDestruidas = crearLabelDato( x: 340, y: 255, ancho: 175, alto: 28);
    lblZombisInvocados = crearLabelDato( x: 640, y: 150, ancho: 175, alto: 28);
    lblCerebrosGastados = crearLabelDato( x: 640, y: 185, ancho: 175, alto: 28);
    lblMejorTiempo = crearLabelDato( x: 640, y: 220, ancho: 175, alto: 28);
    lblPodadoras = crearLabelDato( x: 40, y: 255, ancho: 175, alto: 28);
    for (int i=0;i<5;i++) {
        lblTopRapidos[i] = crearLabelRanking( x: 220, y: 375+i*20, ancho: 160, alto: 22);
        lblTopDestructores[i] = crearLabelRanking( x: 435, y: 375+i*20, ancho: 160, alto: 22);
        lblTopExitosos[i] = crearLabelRanking( x: 675, y: 375+i*20, ancho: 160, alto: 22);
    }
    JButton btnRegresar = new JButton( text: " ");
    btnRegresar.setBounds( x: 375, y: 490, width: 250, height: 45);
    btnRegresar.setOpaque(false); btnRegresar.setContentAreaFilled(false);
    btnRegresar.setBorderPainted(false); btnRegresar.setFocusPainted(false);
    btnRegresar.setCursor(new Cursor(Cursor.HAND_CURSOR));
    btnRegresar.addActionListener( ActionEvent e -> ventana.cambiarPantalla(new MenuInicio(ventana)));
    add(btnRegresar);
}
```

Debido a que usamos un Layout nulo podemos poner la posicion que queramos las cosas lo que nos permite hacerlo mejor visualmente

```java
private void mostrarSinCuenta() {  2 usages  Radhdeat
    lblPartidasJugadas.setText("-"); lblPartidasGanadas.setText("-");
    lblPartidasPerdidas.setText("-"); lblPlantasDestruidas.setText("-");
    lblZombisInvocados.setText("-"); lblCerebrosGastados.setText("-");
    lblMejorTiempo.setText("-"); lblPodadoras.setText("-");
}
```

En caso de que no hayan cuentas creadas mostramos espacios en blanco representados por guiones

```java
package Backend.Cuentas;

// Se crean las cuentas
public class Cuenta {  9 usages  Radhdeat

    // Atributo identificador del usuario
    private String nombrePartida;  3 usages

    // Estadísticas
    private int partidasJugadas;  7 usages
    private int partidasGanadas;  4 usages
    private int partidasPerdidas;  3 usages
    private int plantasDestruidas;  4 usages
    private int zombisInvocados;  3 usages
    private int cerebrosGastados;  3 usages
    // Almacena en segundos
    private int mejorTiempoVictoria;  3 usages
    private int podadorasActivadas;  3 usages

    // Constructor para una cuenta nueva
    public Cuenta(String nombrePartida) {  2 usages  Radhdeat

        // instanciamos todo como 0 ya que es una nueva partida
        this.nombrePartida = nombrePartida;
        this.partidasJugadas = 0;
        this.partidasGanadas = 0;
        this.partidasPerdidas = 0;
        this.plantasDestruidas = 0;
        this.zombisInvocados = 0;
        this.cerebrosGastados = 0;
        this.mejorTiempoVictoria = 0;
        this.podadorasActivadas = 0;
    }
```

Ya que hablamos de cuentas el objeto cuenta es el encargado de almacenar la información de la partida y esta sirve para guardarse despues

```java
public class ManejadorCuentas {  3 usages  Radhdeat *

    // Es una constante que se usa para saber donde se guardaran los datos del objeto
    private static final String DATOS_CUENTA = "cuentas";  8 usages
    // Variable que almacena los datos de la cuenta
    private ListaDinamica<Cuenta> cuentas;  9 usages

    // constructor
    public ManejadorCuentas() {  1 usage  Radhdeat
        // se genera una lista dinamica nueva
        this.cuentas = new ListaDinamica<>();
        File directorio = new File(DATOS_CUENTA);
        // si la el dierctorio no existe llamamos al metodo de mkdirs que crea directorios
        if (!directorio.exists()) {
            directorio.mkdirs();
        }
        // llamamos al metodo para cargar cuentas
        cargarCuentas();
    }
```

El manejador de cuentas es el que usamos para administrar la información de cuentas

```java
// Validamos si el nombre es correcto
public String validarNombre(String nombre) {  1 usage  Radhdeat

    // si no existe un nombre entonces pide el nombre
    if (nombre == null) {
        return "Tienes que colocar un nombre a la partida";
    }

    // Eliminamos los espacios enblanco que dejamos en el nombre
    nombre = nombre.trim();

    // Comprobamos que el nombre tenga al menos una letra y maximo 12
    if (nombre.length() < 1 || nombre.length() > 12) {
        return "El nombre debe tener entre 1 y 12 caracteres.";
    }

    // Quitamos los caracteres especiales
    for (int i = 0; i < nombre.length(); i++) {
        // Descomponemos el nombre por caracteres
        char c = nombre.charAt(i);
        // si el caracter es distinto a letras o numeros
        if (!Character.isLetterOrDigit(c)) {
            return "El nombre solo puede ser Letras y Numeros";
        }
    }
```

Debido a que tenemos normas con los nombres pasamos el nombre por una seria de parametros si los cumple todos entonces guarda el nombre de partida

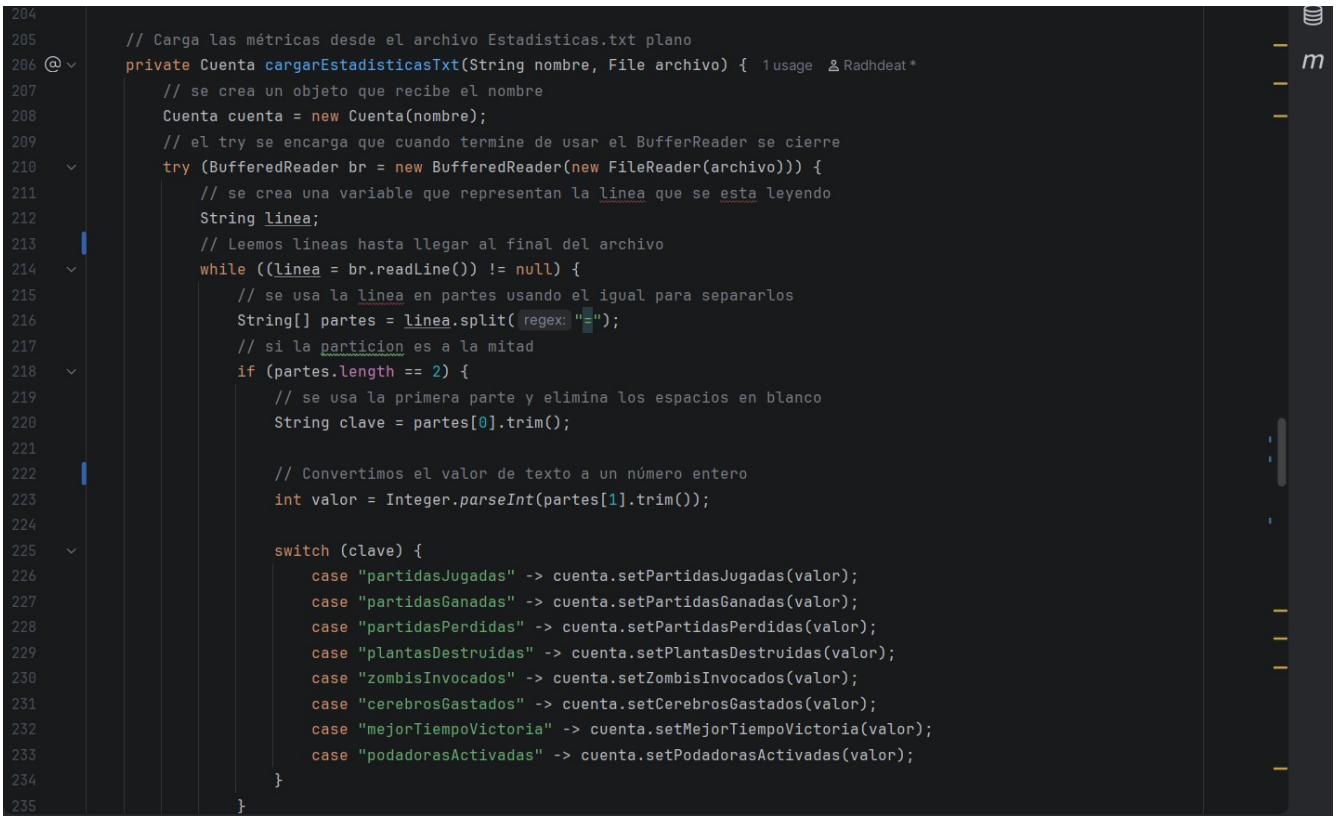
```java
// Cargar todas las cuentas leyendo sus archivos estadisticas
public void cargarCuentas() {  1 usage  Radhdeat
    // se crea una lista dinamica
    cuentas = new ListaDinamica<>();

    // se crea un archivo que representa al directorio en el cual se guardaran los datos
    File directorioBase = new File(DATOS_CUENTA);
    // Se crea un arreglo en el cual recie los archivos sin filtros
    File[] carpetas = directorioBase.listFiles();

    // si la carpeta es nula retorna
    if (carpetas == null) return;

    // recorre las carpetas
    for (File carpeta : carpetas) {
        // si en la entrada es un directorio
        if (carpeta.isDirectory()) {
            // Construye una ruta
            File archivoEstadisticas = new File(carpeta,  child: "Estadisticas.txt");
            // si el archivo existe
            if (archivoEstadisticas.exists()) {
                // llama al metodo que lee las estadisticas
                Cuenta cuenta = cargarEstadisticasTxt(carpeta.getName(), archivoEstadisticas);
                // si la cuenta diferente a nula entonces crea la cuenta
                if (cuenta != null) {
                    cuentas.agregar(cuenta);
                }
            }
        }
    }
```

Tambien contamos con un metodo el cual es el encargado de cargar los datos de las partidas anteriores



```java
// Carga las métricas desde el archivo Estadisticas.txt plano
private Cuenta cargarEstadisticasTxt(String nombre, File archivo) {  1 usage  Radhdeat *
    // se crea un objeto que recibe el nombre
    Cuenta cuenta = new Cuenta(nombre);
    // el try se encarga que cuando termine de usar el BufferReader se cierre
    try (BufferedReader br = new BufferedReader(new FileReader(archivo))) {
        // se crea una variable que representan la linea que se esta leyendo
        String linea;
        // Leemos líneas hasta llegar al final del archivo
        while ((linea = br.readLine()) != null) {
            // se usa la linea en partes usando el igual para separarlos
            String[] partes = linea.split( regex: "=");
            // si la particion es a la mitad
            if (partes.length == 2) {
                // se usa la primera parte y elimina los espacios en blanco
                String clave = partes[0].trim();

                // Convertimos el valor de texto a un número entero
                int valor = Integer.parseInt(partes[1].trim());

                switch (clave) {
                    case "partidasJugadas" -> cuenta.setPartidasJugadas(valor);
                    case "partidasGanadas" -> cuenta.setPartidasGanadas(valor);
                    case "partidasPerdidas" -> cuenta.setPartidasPerdidas(valor);
                    case "plantasDestruidas" -> cuenta.setPlantasDestruidas(valor);
                    case "zombisInvocados" -> cuenta.setZombisInvocados(valor);
                    case "cerebrosGastados" -> cuenta.setCerebrosGastados(valor);
                    case "mejorTiempoVictoria" -> cuenta.setMejorTiempoVictoria(valor);
                    case "podadorasActivadas" -> cuenta.setPodadorasActivadas(valor);
                }
            }
```

En el cual solo se sobre escriben las estadísticas guardadas por las actuales

```java
    // Guardar estadisticas en archivo de texto plano (.txt) por cuenta
    public void guardarEstadisticasTxt(Cuenta cuenta) {  2 usages  Radhdeat *
        // construye una ruta para el archivo
        File archivo = new File( parent: DATOS_CUENTA + "/" + cuenta.getNombrePartida(),  child: "Estadisticas.txt");
        // Abrimos el archivo para escribir estadísticas en formato de texto
        try (BufferedWriter bw = new BufferedWriter(new FileWriter(archivo))) {
            bw.write( str: "partidasJugadas=" + cuenta.getPartidasJugadas());
            bw.newLine();
            bw.write( str: "partidasGanadas=" + cuenta.getPartidasGanadas());
            bw.newLine();
            bw.write( str: "partidasPerdidas=" + cuenta.getPartidasPerdidas());
            bw.newLine();
            bw.write( str: "plantasDestruidas=" + cuenta.getPlantasDestruidas());
            bw.newLine();
            bw.write( str: "zombisInvocados=" + cuenta.getZombisInvocados());
            bw.newLine();
            bw.write( str: "cerebrosGastados=" + cuenta.getCerebrosGastados());
            bw.newLine();
            bw.write( str: "mejorTiempoVictoria=" + cuenta.getMejorTiempoVictoria());
            bw.newLine();
            bw.write( str: "podadorasActivadas=" + cuenta.getPodadorasActivadas());
            // try por en caso de que exista un error de escritura
        } catch (IOException e) {
            System.err.println("Error al guardar la estadistica de " + cuenta.getNombrePartida() + ": " + e.getMessage());
        }
    }
```

Para guardar las estadisticas solo usabor un BufferWriter que basicamente escribe cadenas lineas de texto y los escribe en el archivo que creamos

```java
//clase para leer los archivos Json
public class LectorNivelJson {  3 usages  Radhdeat

    // declaramos atributos
    private int duracionSegundos;  2 usages
    private int cerebrosIniciales;  2 usages
    private String[][] jardin;  3 usages

    // metodo que recibe la ruta del archivo
    public boolean cargarConfiguracion(String rutaArchivo) {  1 usage  Radhdeat
        // Se crea un StringBuilder vacío para almacenar el contenido del archivo
        StringBuilder sb = new StringBuilder();
        // se lee el archivo completo
        try (BufferedReader br = new BufferedReader(new FileReader(rutaArchivo))) {
            // Se declara una variable para almacenar cada línea del archivo
            String linea;
            // repite la lectura mientras hayan lineas
            while ((linea = br.readLine()) != null) {
                // Elimina espacios al inicio y final y agrega la línea al StringBuilder
                sb.append(linea.trim());
            }
            // por en caso de que haya un error en la lectura
        } catch (IOException e) {
            System.err.println("Error al leer el archivo JSON: " + e.getMessage());
            return false;
        }
```

Lector Json es uno de los mas importantes ya que este es el encargado de interpretar el Json para porderlo usar, para no usar librerias externas creamos una cadena de texto vacia

ya que esta guardara la informacion, despues de leerla borra los espacios en blanco que tiene al inicio y al final,

```
// Convierte el contenido del StringBuilder en una cadena de texto
String contenido = sb.toString();

// Extraer propiedades principales
this.duracionSegundos = extraerEntero(contenido,  clave: "duracionSegundos");
this.cerebrosIniciales = extraerEntero(contenido,  clave: "cerebrosIniciales");

//Extraer y validar el arreglo del jardín
this.jardin = extraerJardinNativo(contenido);
if (this.jardin == null) {
    return false;
}

return true;
```

Después de esto instancia la información que le solicitamos como lo es la duración de los segundo y los cerebros iniciales

```
93 @    private String[][] extraerJardinNativo(String json) {  1 usage   Radhdeat
94
95          //Busca la posición donde comienza la clave "jardin".
96          int posJardin = json.indexOf("\"jardin\"");
97          if (posJardin == -1) return null;
98
99          // Busca el primer corchete de apertura después de esa clave.
100         int inicioCorchete = json.indexOf( str: "[", posJardin);
101         if (inicioCorchete == -1) return null;
102
103         // Contar la profundidad de corchetes para hallar su cierre exacto
104         int corchetesAbiertos = 0;
105         int finCorchete = -1;
106
107         // For que recorre Despues del corchete hasta donde esta el ultimo corchete
108         for (int i = inicioCorchete; i < json.length(); i++) {
109             // Obtiene el carácter que se encuentra en la posiscion i
110             char c = json.charAt(i);
111
112             if (c == '[') {
113                 corchetesAbiertos++;
114             } else if (c == ']') {
115                 corchetesAbiertos--;
116                 if (corchetesAbiertos == 0) {
117                     finCorchete = i;
118                     break;
119                 }
120             }
121         }
```

Después de extraemos la información que esta dentro de los corchetes y esta la guardamos

```java
// Verifica si el identificador pertenece a una planta permitida o a VACIO
private boolean esIdentificadorValido(String id) {  no usages  Radhdeat
    return id.equals("VACIO") ||
            id.equals("LANZAGUISANTES") ||
            id.equals("PATATAPUM") ||
            id.equals("HIELAGUISANTES") ||
            id.equals("NUEZ") ||
            id.equals("CARRONIVORA") ||
            id.equals("PINCHOHIERBA") ||
            id.equals("BIPETIDORA");
}
```

Para interpretarlas después.

```java
// Usamos T que indica que es un tipo generico para almacenar diferentes tipos de objetos
public class ListaDinamica<T> {  18 usages  Radhdeat

    // Atributos
    private Object[] elementos;  11 usages
    private int tamaño;  11 usages
    private static final int CAPACIDAD_INICIAL = 10;  1 usage

    // constructor
    public ListaDinamica() {  6 usages  Radhdeat
        this.elementos = new Object[CAPACIDAD_INICIAL];
        this.tamaño = 0;
    }

    // Agrega un elemento al final de la lista. Si el arreglo se llena, duplica su capacidad.
    public void agregar(T elemento) {  4 usages  Radhdeat
        // si los elementos estan llenos
        if (tamaño == elementos.length) {
            // llama al metodo para redimensionar
            redimensionar();
        }

        elementos[tamaño] = elemento;
        tamaño++;
    }
```

Lista dinámica es otra clase muy importante ya que esta nos permite tener un arreglo dinámico sin necesidad de usar un ArrayList

```java
    // Método privado auxiliar para duplicar el tamaño del arreglo interno cuando se llena
    private void redimensionar() {  1 usage  Radhdeat
        // se crea otro arreglo pero con el doble de espacio
        Object[] nuevoArreglo = new Object[elementos.length * 2];
        // recorre el arreglo de elementos
        for (int i = 0; i < elementos.length; i++) {
            // y copia lso datos en el nuevo arreglo
            nuevoArreglo[i] = elementos[i];
        }
        elementos = nuevoArreglo;
    }
}
```

Ya que si el arreglo se llena nosotros creamos otro con el doble de capacidad y copiamos la información previa y la guardamos en el nuevo arreglo

Instalación del entorno de desarrollo

Para instalar el archivo es necesario el uso de un IDE. Como NeatBeans ya que hay una carpeta llamada Proyecto 2 que también contiene el manual técnico, manual de usuario y la carpeta del archivo.
En este caso para ejecutarlo hay que acceder a consola y escribir este comando java -jar Proyecto2.jar el cual ejecutara el juego pero si quieres acceder al código fuente hay que importar toda la carpeta, luego ya solo ver que parte del código se quiere